

Reimplementing and Improving Neuro-symbolic AI in LLM Logic

AIEA Lab Final Report

Rami Al Habash, 3/17/2026

Dr. Leilani Glippin

Introduction

Neuro-symbolic AI is one of the primary ways to reduce AI hallucinations and acquire more accurate results. LLMs are known to hallucinate specifically with tasks that require symbolic reasoning, like logic tasks. The idea is not to rely fully on AI for these types of questions and only use it for what they are good for, which is things like translation. For example, when an LLM is tasked with proving a hypothesis for a logic-type question, we task the LLM to translate it into First Order Logic (FOL) and then use a theorem prover to answer the question. This report will cover the type of logic questions we're looking to solve, different approaches to solving them, and how we can reimplement and improve the neuro-symbolic approach.

Logic Datasets

The research I looked into covers the example problem in the introduction: How do we make AI more reliable at solving logic-based problems? Let's first take a look at what logic-based problems actually are. There are two types of important datasets that many research papers in this field use to benchmark test their implementations: ProofWriter (PW) (Tafjord et al., 2021) and FOLIO (Han et al., 2022).

```
<premises>
Coffee is a drink.
Coffee is cold.
If a drink is cold, then it is raining.
<conclusion>
It is raining.
<label>
True
```

Fig. 1: ProofWriter (PW) Sample

```
<premises>
Coffee is a drink.
Coffee is cold.
If a drink is cold, then it is raining.
<conclusion>
It is raining.
<premises-FOL>
Drink(coffee)
Cold(coffee)
 $\forall x (\text{Drink}(x) \wedge \text{Cold}(x) \rightarrow \text{Raining})$ 
<label>
True
```

Fig. 2: FOLIO sample

Let's take a more in-depth look at what these datasets look like. Let's start with Fig. 1, PW. PW is synthetic data. PW has natural language premises, a conclusion, and a label. The premises are natural language facts and rules. The conclusion is a query that follows from the premises. The label is either True, false, or uncertain. The ProofWriter dataset comes with many

similar examples to Fig. 1 of differing complexity. Models are usually tested on how accurately they can predict the label.

Moving on to Fig. 2, FOLIO is similar to ProofWriter. However, it has an added section called premises-FOL. Unlike ProofWriter, FOLIO is written by humans and is engineered to be a more difficult benchmark that stress tests models.

Logic-solving research

Many researchers have sought out different ways to solve this logic dilemma. We will cover two influential research papers in the field that jump-started many others. Chain of Thought (COT) (Wei et al. 2022) and LINC (Olausson et al., 2023). They both have their ups and downs, and they both approach symbolic reasoning differently.

Let's first talk about COT. COT takes a neural approach to solving symbolic reasoning problems. COT prompts the LLM to think step by step before saying its final answer. Explicitly telling the LLM to "think out loud" or to write its reasoning step by step has proven to be more effective when it comes to symbolic reasoning. The upside to COT is that it is easier to set up because it relies heavily on prompting the LLM to think step by step and predict the label for problems like those in FOLIO and PW datasets.

LINC focuses more on a neuro-symbolic approach. LINC uses the LLM as a mere semantic parser. Essentially, translating natural language premises to FOL premises. LINC shows the LLM a few-shot examples of premises and premises-FOL from Fig. 2, and then prompts the LLM to translate premises into FOL. Once they're in FOL, they are then passed to a theorem prover, specifically Prover9, which uses logical inference to prove the conclusion either true, false, or uncertain.

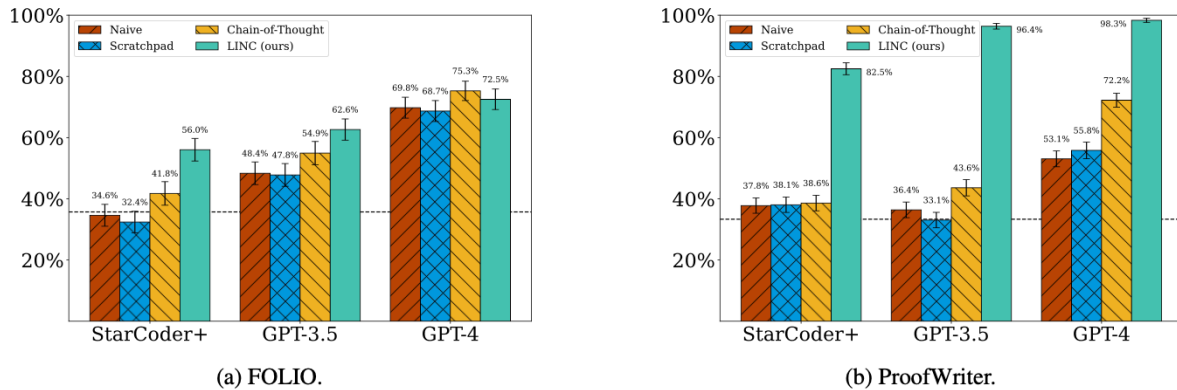


Figure 3: Results of each model on the FOLIO and ProofWriter datasets. Accuracies are for bootstrapped 10-way majority vote for all models. Error bars are ± 1 bootstrapped standard deviation. Dotted, black line is the accuracy obtained by always guessing the most common label in the dataset.

Fig. 3 LINC's results on FOLIO and ProofWriter (Olausson et al., 2023)

Fig 3. Shows us that the neuro-symbolic approach, like LINC's, performs far better than other approaches. Neuro-symbolic AI is a lot more foolproof than other methods and is a top contender for logic-based problems. However, I think it's important to note that COT is second best. The authors of LINC would likely argue that no neural approach would be as good as using

a theorem prover. While COT authors would likely not disagree, they would stress the importance of the generality and simplicity of making the model itself a better logical solver.

Reimplementing Neuro-symbolic AI

I decided to gain hands-on experience re-implementing neuro-symbolic approaches. I reimplemented LINC to start with. For starters, I just wanted to see if I could get their environment set up and get things up and running. I created a small knowledge base in Prolog about soccer players and the rules about the leagues they play in. I then used an LLM to generate 50 PW examples from my Prolog knowledge base and ran LINC's code on them. Alongside this, I ran each of 50 actual PW and FOLIO examples. Basically, I ran a smaller scale of their study using much simpler models and a subset of their dataset and my own dataset.

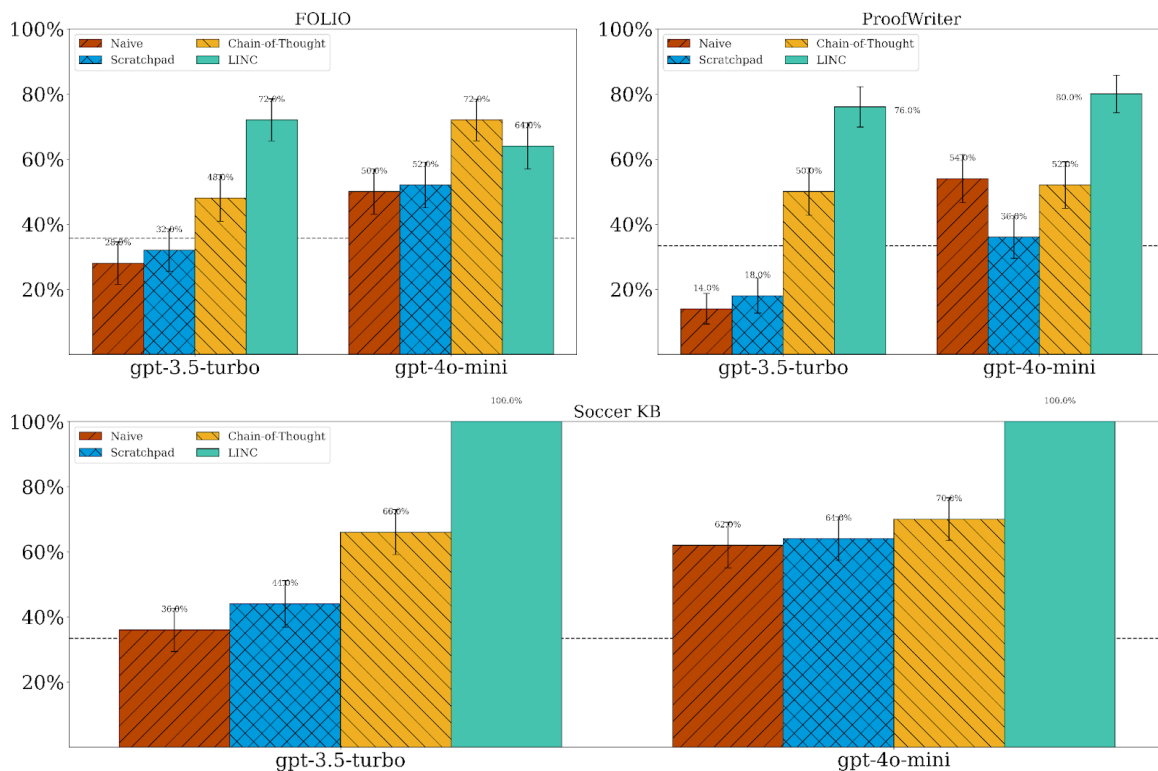


Fig. 4. Mini LINC reimplementations

My small-scale experiment is fairly similar to LINC's actual study. I will continue to use my Prolog soccer KB to make some more experiments on LINC. However, this was my first look at seeing how LINC works and how to use their code. I also took a look and understood a lot of their functions

In-depth Reimplementation and Experimentation

I wanted to get a more in-depth understanding of LINC's codebase and format it in my own way, and add more to it. For starters, I reimplemented LINC using LangChain. I slowly started creating my own codebase independent of LINC's. Even though I used many functions from their codebase. Learning about LangChain and how to use it excessively simplifies AI prompting. LangChain offers a wide variety of simple functions for prompting models, calling them, and parsing their output (aka a chain). I also employed RAG retrieval on LINC's codebase.

I used FAISS RAG retrieval. Basically, I retrieve relevant examples from my knowledge base to the example the LLM is asked to solve.

The hardest part of this task was navigating and finding all the relevant functions and actually recreating LINC using LangChain. At the time of completing this task, my reimplementaion flowed as follows:

LINC: ProofWriter $\rightarrow$ Semantic Parser (LLM) $\rightarrow$ FOL $\rightarrow$ Prover9 $\rightarrow$ verdict

LINC+ (my pipeline): Prolog $\rightarrow$ LLM $\rightarrow$ ProofWriter $\rightarrow$ RAG $\rightarrow$ Semantic Parser (LLM) $\rightarrow$ Prover9 $\rightarrow$ Prover9 trace and results.

I applied my RAG retrieval to my LLM-generated ProofWriter dataset. The issue with this is that it didn't provide the parser any help with parsing. Rather, expanding its domain. This was the largest flaw in this step that I aimed to improve. However, this step was significant because now I had my own codebase that is similar to LINC's, which uses LangChain and RAG and still produced impressive results.

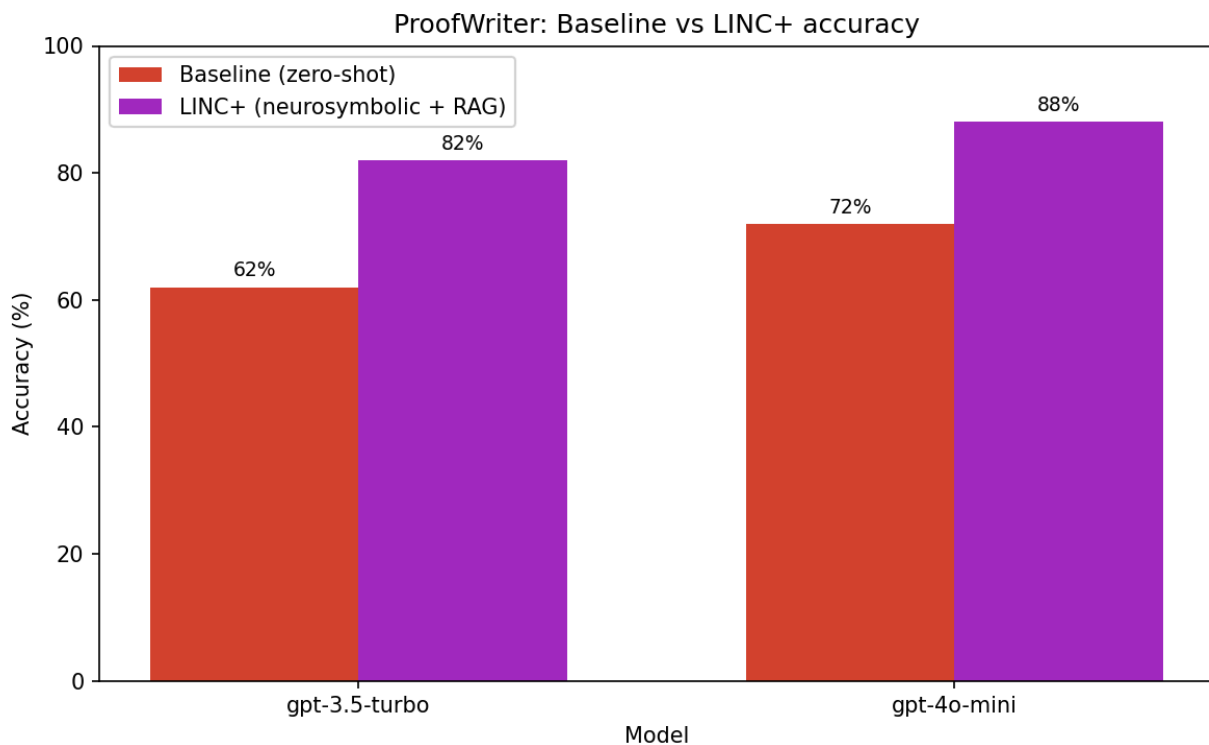


Fig. 5 LINC+ Benchmark Results

Fig. 5 shows the benchmark test I ran on LINC+; it predicted the label quite well and performed better than the baseline model. I would keep referencing this benchmark test on my generated data to see if I could make more improvements. Now I officially have a LINC codebase to alter the way I like, and also I made it more interpretable. I created a way to test queries/conclusions and see what goes on in the program as it evaluates. For example, you can see the conclusion, the predicted label, the actual label, the FOL translation, and Prover9's inference trace. Making it more interpretable allowed me to better see what goes on behind the scenes and pushed me to make more educated improvements.

Improving Neuro-symbolic AI

I wanted to make my structure more interpretable and my RAG more relevant in LINC+. This new wave of improvement I called “myLINC.” To make LINC+ more interpretable, I decided to use LangGraph to break down the pipeline into nodes. I also wanted to see what goes on in each node, so I learned how to use LangSmith. LangSmith allows you to see what goes on in each node and what the LLMs that are called using LangGraph output. LangGraph was an extremely nice addition that I will be using in the future.

To make RAG more relevant, I decided to come up with a different pipeline using nodes in LangGraph. This time, I wanted my program to retrieve relevant translations from FOLIO rather than PW because, as we discussed earlier, FOLIO directly shows the LLM how to translate natural language to FOL. This time, my pipeline looked a lot more like this:

- Setup:

Prolog file → LLM → ProofWriter and FOLIO-type dataset

- For each call to predict a label graph:

ProofWriter/FOLIO → RAG FOLIO → LLM judge → if relevant → semantic parser → Prover9
 |
 Else → COT

- run_test: runs the benchmark tests on all my LLM-generated ProofWriter/FOLIO data

- run_queries: runs some queries on my data, showing detailed logical inference traces and such

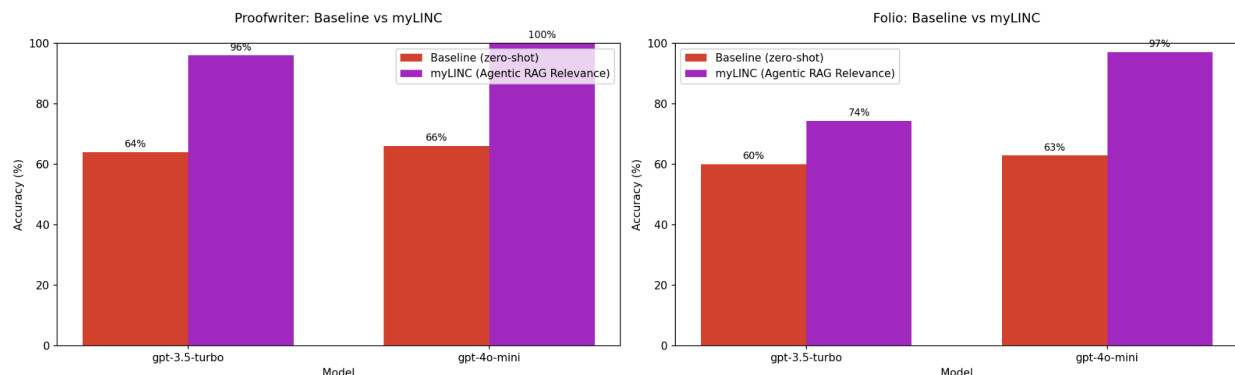


Fig. 6 myLINC Benchmark Results

Fig. 6 clearly looks better than Fig. 5 on Proofwriter. We are seeing large improvements here in model translation. I think this is because I am using RAG on FOLIO instead of ProofWriter, and not because my relevance judge is using the COT fallback. I know this because when I ran the queries, the program always redirected to LINC. The judge always found the RAG information to be relevant. This is not a bug but rather an indicator that the pipeline is working well because, almost always, the RAG examples of FOLIO are relevant for LLM translation.

myLINC would be great to deploy on a larger scale to test it. However, I found similar research by researchers doing RAG retrieval on LINC, so I don't know how much of a viable path this is, but it might be worth exploiting.

Conclusion

I learned a lot about LLMs in logic throughout my tasks at the AIEA lab. LLMs are not the most reliable at logical inference and symbolic reasoning. Studies like LINC show how neuro-symbolic reasoning shines in comparison to naive LLMs and COT implementations. However, I must add that I am impressed by COT because it's simple, shows the strength of prompting, and introduces the potential of LLM logical inference. By that, I mean LLMs thinking step by step. I believe further research could be done to improve LLM inference. However, for now, and for a while, using a more symbolic approach and relying on neuro-symbolic reasoning is more sound.

References

- Han, S., et al. (2022). *FOLIO: Natural language reasoning with first-order logic*. arXiv:2209.00840. <https://doi.org/10.48550/arXiv.2209.00840>
- Olausson, T., Gu, A., Lipkin, B., Zhang, C., Solar-Lezama, A., Tenenbaum, J., & Levy, R. (2023). LINC: A neurosymbolic approach for logical reasoning by combining language models with first-order logic provers. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 5153–5176. <https://doi.org/10.18653/v1/2023.emnlp-main.313>
- Tafjord, O., Dalvi Mishra, B., & Clark, P. (2021). *ProofWriter: Generating implications, proofs, and abductive statements over natural language*. Findings of ACL 2021. <https://doi.org/10.48550/arXiv.2012.13048>
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Chi, E. H., Le, Q., & Zhou, D. (2022). *Chain of thought prompting elicits reasoning in large language models*. arXiv:2201.11903. <https://arxiv.org/abs/2201.11903>